

# Python Comprehensive Problem Set: Basics, OOP & Libraries

## Advanced Programming & Problem Solving Assessment

Practice Problems covering Python Basics, Object-Oriented Programming, and Standard Libraries

### Part 1: Python Basics & Data Structures

#### Problem 1: List Operations and Filtering

Write a Python program that takes a list of integers from the user, filters out all odd numbers, and returns a new list containing only the even numbers multiplied by 2 using a List Comprehension.

#### Problem 2: String Manipulation and Palindrome Check

Write a function named `is_palindrome(text: str) -> bool` that checks whether a given string is a palindrome (reads the same forwards and backwards), ignoring spaces and capitalization. Do not use built-in reverse slicing (`[::-1]`) directly inside the return statement; implement a loop or standard approach.

#### Problem 3: Dictionary Frequency Counter

Given a sentence as a string, write a program that counts the frequency of each word using a dictionary. Ignore punctuation and treat words case-insensitively.

```
sentence = "Learn Python programming, Python is powerful and easy to learn."
```

## Part 2: Object-Oriented Programming (OOP)

---

### Problem 4: Bank Account Management System

Create a class named `BankAccount` with the following specifications:

- Attributes: `account_holder` (string) and `balance` (float, default to 0.0).
- Method `deposit(amount)`: Adds the amount to the balance if it's positive; otherwise, prints an error message.
- Method `withdraw(amount)`: Subtracts the amount if sufficient balance exists; otherwise, prints "Insufficient funds!".
- Method `get_balance()`: Returns the current balance.

### Problem 5: Inheritance and Polymorphism

Create a base class named `Shape` with an empty method `area()`. Then create two subclasses: `Rectangle` (taking width and height) and `Circle` (taking radius). Override the `area()` method in both subclasses to return the correct mathematical area. Test your classes by creating a list of mixed shapes and printing their areas.

### Problem 6: Library Item Hierarchy

Design an OOP structure for a library system. Create a parent class `LibraryItem` with attributes `title` and `item_id`. Create a subclass `Book` that inherits from `LibraryItem` and adds an attribute `author` and a method `get_details()` that returns a formatted string containing all item details.

## Part 3: Standard Libraries (Math, Random, Datetime, Collections)

---

### Problem 7: Random Password Generator (Random Module)

Write a Python program using the `random` module that generates a secure random password of length 12. The password must contain a mix of uppercase letters, lowercase letters, digits, and punctuation symbols.

**Problem 8: Age Calculator in Days (Datetime Module)**

Write a function that takes a birthdate in the format YYYY-MM-DD as a string, uses the datetime module to calculate the exact age of the person in days up to the current date, and returns the total number of days lived.

**Problem 9: Inventory Record using NamedTuple (Collections Module)**

Use namedtuple from the collections module to create a structure named Product with fields: name, price, and quantity. Create a list of three products, and write a loop to calculate and print the total inventory value (price × quantity) for each.

**Problem 10: Most Common Elements (Collections Module)**

Given a list of student grades (e.g., `['A', 'B', 'A', 'C', 'B', 'A', 'D', 'B', 'B', 'A']`), use Counter from the collections module to find and print the most common grade and its exact frequency count.